

布隆过滤器

布隆过滤器（Bloom Filter）是一种用于判断元素是否属于某个集合的概率数据结构，由 Burton Howard Bloom 于 1970 年提出。[Bloom, 1970] 它通过一系列散列函数将元素映射到二进制位数组中，具有很高的空间效率，查询时间效率也远超一般算法。不过，它的判断可能产生假阳性（误报），不会产生假阴性（漏报），而且简单布隆过滤器难以删除元素。

判断元素是否属于集合，常见的做法是先保存集合中的全部元素，再通过比较来检索。链表、树和散列表（也叫哈希表）都采用这种思路，检索时间复杂度分别为 $O(n)$ 、 $O(\log n)$ 和 $O(1)$ 。随着元素数量增加，保存整个集合需要的空间会越来越大，检索也会面临速度上的负担。布隆过滤器则用位数组记录元素经过散列映射后的位置，不需要保存元素本身。

一个空布隆过滤器包含长度为 m 的位数组，所有位的初始值都是 0，并配有 k 个不同的散列函数。每个函数都将元素映射到数组的 m 个位置之一。为了达到最佳效果，这些函数应当均匀分布且相互独立。通常， k 是一个小常数，取决于期望的假阳性率 ϵ ； m 则与 k 和准备添加的元素数量成正比。

添加元素时，分别用 k 个散列函数计算它在数组中对应的 k 个位置，并将这些位置的位全部设为 1。查询时也计算相同的 k 个位置：只要其中有一位是 0，就能确定该元素不在集合中，因为已经添加的元素会将所有对应位置设为 1；如果这些位全是 1，则只能判断该元素可能在集合中。其他元素也可能恰好将这些位置全部设为 1，从而造成假阳性。简单布隆过滤器无法区分这两种情况，但更先进的技术可以解决这一问题。

例如，用 $m = 18$ 、 $k = 3$ 的布隆过滤器表示集合 $\{x, y, z\}$ 。示意图中的彩色箭头表示各元素经过散列函数映射到数组中的位置，箭头个数对应散列函数的数量。元素 w 的散列结果中有一个位置的位为 0，因此可以确定 w 不在集合 $\{x, y, z\}$ 中。

当 k 较大时，设计 k 个不同且独立的散列函数可能很困难。如果一个良好的散列函数具有较宽的输出，其不同位域之间通常几乎没有相关性，便可以把输出切分成多个位域，用来生成多个“不同”的散列函数。另一种做法是向支持初始值的散列函数传入 k 个不同的初始值，例如 $0, 1, \dots, k - 1$ ；也可以将这些值加入或追加到键中。对于较大的 m 和/或 k ，可以放宽散列函数之间的独立性要求，由此增加的假阳性率可以忽略不计。[Dillinger 与 Manolios, 2004a; Kirsch 与 Mitzenmacher, 2006] 增强双重散列和三重散列也能有效地导出 k 个索引。[Dillinger 与 Manolios, 2004b] 这两种方法都是双散列的变体，实际上是以两个或三个散列值作为种子的简单随机数生成器。

简单布隆过滤器无法安全地删除元素，因为它不能判断一个元素对应的 k 个位中，哪些位还被其他元素使用。将其中任意一位清零，就足以让查询不再认定该元素可能存在，但也可能使其他恰好映射到这一位的元素被判定为不存在。简单算法没有记录这些位由哪些元素共同设置，因此清除任何一位都有引入假阴性的可能。

若要模拟一次性删除，可以另外建立一个辅助布隆过滤器，称为“移除过滤器”，专门记录已移除的元素。但移除过滤器也会产生假阳性：它可能把未移除的元素误判为已移除，使原过滤器与移除过滤器组成的复合过滤器产生假阴性。这种方法还无法重新添加先前移除的元素，因为重新添加前必须先将它从移除过滤器中删除，又回到了同一个难题。

布隆过滤器在空间和时间方面都有很大优势。建好后的位数组占用固定的存储空间；插入和查询的时间复杂度都是 $O(k)$ ，当 k 为小常数时，所需时间也是常数级。各散列函数的计算互不依赖，便于通过硬件并行实现。由于不必存储元素本身，它在某些保密要求非常严格的场合也有优势。

假阳性率是使用布隆过滤器时需要权衡的不足：存入的元素越多，假阳性率就越高。如果元素数量很少，使用散列表就已足够。围绕降低假阳性率的研究，也产生了许多布隆过滤器的变种。在许多场景中，所有待过滤的键都可以取得，但逐一枚举它们的代价较高，例如需要多次读取硬盘。当假阳性率变得过高时，可以重新生成过滤器，不过这类重建应当相对少见。